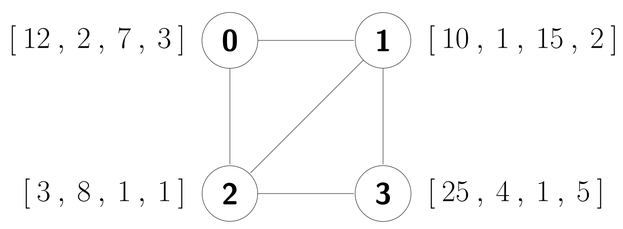
Translation from MPLang to MPNN

Nette Beckers,
Prof Dr. Jan Van den Bussche,
Juno Steegmans

Graphs & feature maps

Graph $G = (V, E)$, with V the set of nodes and E the set of edges.
A node has d features, the function χ maps the feature vector to their node.
Example graph with features:



Global Feature Map Transformer

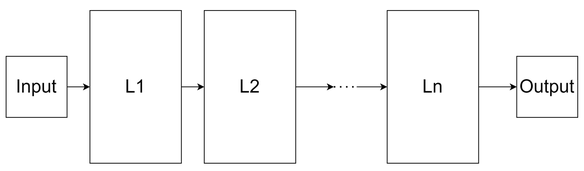
Goal: uniform functions on feature maps.

GFMT of type $d \to r$, transforms input feature map on the graph with d features per node into a feature map with r features per node.
Example: $T_1(G)(\chi)(v) = (\chi(v)[1] + \chi(v)[2])/2$ of type $2 \to 1$, sum of 2 features.

Operations: concatenation $T_1|T_2$, parallel composition $T_1||T_2$, sequential composition $T_1; T_2$, sum $T_1 + T_2$ and difference $T_1 - T_2$.

Message Passing Neural Network

Subclass of **Graph Neural Networks (GNN)**
Sequence of layers, $L_1; L_2; \ldots; L_n$



One layer, $L = (W_1, W_2, b, \sigma)$

Calculation of 1 layer

$$L(G)(\chi)(v) := \sigma(W_1\chi(v) + W_2 \sum_{u \in N(G)(v)} \chi(u) + b)$$

MPLang

$$e ::= a \mid P_i \mid a \cdot e \mid e + e \mid f(e) \mid \Diamond e$$
With $a \in \mathbb{N}^0$, $i \in \mathbb{R}$ and $f : \mathbb{R} \to \mathbb{R}$

Represent GFMT of type $d \to 1$

- if $e = a$, then $e(G)(\chi)(v) := a$
- if $e = P_i$, then $e(G)(\chi)(v) :=$ the i-th component from $\chi(v)$
- if $e = ae_1$, then $e(G)(\chi)(v) := ae_1(G)(\chi_1)(v)$
- if $e = e_1 + e_2$, then $e(G)(\chi)(v) := e_1(G)(\chi_1)(v) + e_2(G)(\chi_2)(v)$
- if $e = f(e_1)$, then $e(G)(\chi)(v) := f(e_1(G)(\chi)(v)$
- if $e = \diamond e_1$, then $e(G)(\chi)(v) := \sum_{u \in N(G)(v)} e_1(G)(\chi)(u)$

Translation MPLang to ReLu-MPNN

- If $e = a$, then $E = (\vec{0}, \vec{0}, a, id)$
- If $e = P_i$, then $E = (\vec{\cdot}, \vec{0}, 0, id)$
- If $e = a \cdot e_1$, then $E = E_1; (a, 0, 0, id)$
- If $e = e_1 + e_2$, then $E = (E_1|E_2); ([1\ 1], [0\ 0], 0, id)$
- If $e = f(e_1)$, then $E = E_1; (1, 0, 0, f)$
- If $e = \Diamond e_1$, then $E = E_; (0, 1, 0, id)$

With E_1 and E_2 the MPNN from subexpressions e_1 and e_2

Algorithm

For layers $L = (W_{1L}, W_{2L}, b_L, \sigma)$
and $K = (W_{1K}, W_{2K}, b_K, \sigma)$ we can:

Concatenation layers

$$(L||K) = (\begin{bmatrix} W_{1L} \\ W_{1K} \end{bmatrix}, \begin{bmatrix} W_{2L} \\ W_{2K} \end{bmatrix}, \begin{bmatrix} b_L \\ b_K \end{bmatrix}, \sigma)$$

Parallel composition layers

$$(L||K) = (\left[\begin{array}{c|c} W_{1L} & 0 \\ \hline 0 & W_{1K} \end{array}\right], \left[\begin{array}{c|c} W_{2L} & 0 \\ \hline 0 & W_{2K} \end{array}\right], \begin{bmatrix} b_L \\ b_K \end{bmatrix}, \sigma)$$

ReLu-MPNN

Every layer ReLu as activation function, except the last one has an id-function.

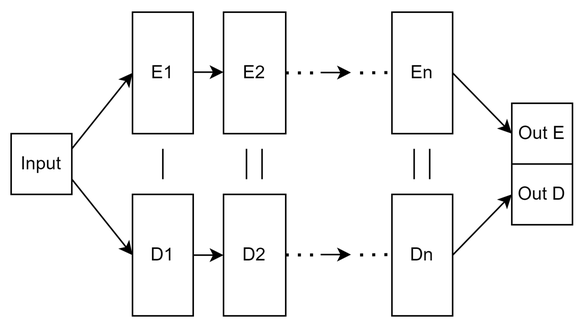
$$M = L_1; L_2; \ldots; L_{n-1}; L_n$$
With L_{n-1} id-layer and L_n σ-layer

$$L_{n-1} = (\begin{bmatrix} W_1 \\ -W_1 \end{bmatrix}, \begin{bmatrix} W_2 \\ -W_2 \end{bmatrix}, \begin{bmatrix} b \\ -b \end{bmatrix}, ReLu)$$

$$L_n = (\left[W_1 \mid -W_1\right], \left[W_2 \mid -W_2\right], b, \sigma)$$

Concatenation ReLu-MPNN's

$$E = E_1; E_2; \ldots; E_n,$$
$$D = D_1; D_2; \ldots; D_n$$



Implementation

Numpy: zeros, fill, concatenate, fill, hstack, vstack
Structure: activation function, graph,
MPNN (MPNN layer, MPNN operator),
MPLang expression (all expressions derived)

MPNN operator

- transform into ReLu-MPNN
- concatenate 2 ReLu-MPNN's

```
def parComp(layer1, layer2) -> MPNNLayer:
    w11 = layer1.getw1() #dim = axb
    w12 = layer2.getw1() #dim = cxd
    w21 = layer1.getw2() #dim = exf
    w22 = layer2.getw2() #dim = gxh
    return MPNNLayer(
        np.vstack(
            np.hstack(w11, np.zeros(a, d)),
            np.hstack(np.zeros(c, b),w12)),
        np.vstack((
            np.hstack(w21, np.zeros(e, h)),
            np.hstack(np.zeros(g,f),w22)),
        np.hstack((
            layer1.getBias(),
            layer2.getBias())),
        layer1.getSigma())
```

Expression $e = 5 + (5 + P_2)$

Example

```
exp1 = number(5)
exp2 = component(2)
exp3 = scalairProduct(exp2, 5)
exp4 = sum(exp1, exp2)
exp5 = function(activationFunction("id"), exp2)
exp6 = diamond(exp4)
exp7 = sum(exp1, exp4)
exp8 = diamond(scalairProduct(exp7, 2))
```

Layer 1 $(\begin{bmatrix} 0&0&0&0 \\ 0&0&0&0 \\ 0&0&0&0 \\ 0&1&0&0 \\ 0&0&0&0 \\ 0&-1&0&0 \end{bmatrix}, \begin{bmatrix} 0&0&0&0 \\ 0&0&0&0 \\ 0&0&0&0 \\ 0&0&0&0 \\ 0&0&0&0 \\ 0&0&0&0 \end{bmatrix}, \begin{bmatrix} 5 \\ -5 \\ 5 \\ 0 \\ -5 \\ 0 \end{bmatrix}, ReLu)$ **Output** $\begin{bmatrix} \begin{bmatrix} 5 \\ 0 \\ 5 \\ 2 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 0 \\ 5 \\ 1 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 0 \\ 5 \\ 8 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 0 \\ 5 \\ 4 \\ 0 \\ 0 \end{bmatrix} \end{bmatrix}$

Layer 2 $(\begin{bmatrix} 1&-1&0&0&0&0 \\ 0&0&1&1&-1&-1 \\ -1&1&0&0&0&0 \\ 0&0&-1&-1&1&1 \end{bmatrix}, \begin{bmatrix} 0&0&0&0&0&0 \\ 0&0&0&0&0&0 \\ 0&0&0&0&0&0 \\ 0&0&0&0&0&0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, ReLu)$ **Output** $\begin{bmatrix} \begin{bmatrix} 5 \\ 7 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 6 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 13 \\ 0 \\ 0 \end{bmatrix} & \begin{bmatrix} 5 \\ 9 \\ 0 \\ 0 \end{bmatrix} \end{bmatrix}$

Layer 3 $(\left[1\ 1\ -1\ -1\right], \left[0\ 0\ 0\ 0\right], 0, id)$ **Output** $\left[12\ 11\ 18\ 14\right]$

nette.beckers@student.uhasselt.be

UHASSELT